Read the Crossings, Compute the Knot: Knot Recognition from Images that Knows When to Abstain

Anonymous CVPR submission

Paper ID *****



Figure 1. **Read the diagram, compute the knot, abstain when the image is not enough.** Green circles: crossings read with over/under probability > 0.9; yellow: the predicted strand centre-line. A real render and a benchmark cable are identified exactly. On a silhouette the over/under reading disagrees across test-time views, so the type probability stays below 0.95 and the system abstains. Where two strands touch, a crossing loses an arm and the system abstains rather than guess.

Abstract

Recognising a knot in an image means reading a planar diagram (where the strands cross and which strand is on top) and then computing its topology. Image-to-label classifiers skip the diagram: they cannot name a knot outside their training census and cannot say when an image does not show enough to decide. We split the problem. A dense network predicts diagram structure (crossing centres, the over-strand axis, cable masks and strand centre-lines); a geometric parser turns the maps into a diagram, and exact knot invariants identify it. Checks that do not trust the network (an Euler count of faces, a four-arm test at every crossing, agreement across test-time views and across two centre-line resolutions) turn likely errors into abstentions. The formulation is backbone-agnostic: with a pair of standard U-Nets it answers 86% of photo-like images on KnotBench-Photo (74% on unseen backgrounds) with no wrong answer and abstains on every silhouette, where a 250-class image classifier is right on 12% and a segmentation network alone can name nothing. On a blind set of 308 drawings and renders from sources never used during development, scored once, it is right on 129 images and wrong on 9; the errors are drawings whose under-strand gaps are one or two pixels wide. A U-Net with a transformer bottleneck, trained on the same data, beats the U-Net pair on a second blind set at every one of three seeds (110.7 vs 101.3 correct of 312, on average) and on real pictures, with about as many wrong answers. We release the benchmark, the blind protocol, an annotation tool for real cable photographs and all code.

1. Introduction

A tangled cable, a surgical suture, a rope in a robot gripper, a knotted protein [23] and a knot drawing in a textbook pose the same question: *which knot is this?* The answer is topological and exact, but it has to be read from pixels. Two facts make the reading hard. First, the decisive evidence is local and occluded: at each crossing one strand passes over the other, and a single misread crossing can change the knot. Second, many images do not contain the answer at all. A silhouette hides over/under, and two strands that touch can

hide a crossing. A useful system must therefore know when not to answer.

Prior learned approaches predict a label from the whole image, such as a knot type from a fixed census or a crossing number [7, 13]. Such a model cannot name a knot it was not trained on, cannot explain its answer and returns a label even for a silhouette. Classical pipelines skeletonise the image and search for junctions; on our benchmark the skeleton parser finds only 0.16 of crossings and guesses over/under at chance (0.49).

We instead *learn perception and compute topology* (Fig. 1). A dense network predicts the parts a diagram is made of. A parser assembles them into a planar diagram with geometric crossing signs, and exact invariants (SnapPy, knot Floer homology) name the knot or link. The structure also yields checks that do not trust the network. For a 4-valent planar projection the number of enclosed faces equals crossings plus connected pieces, so a missed crossing leaves an unexplained face. Every crossing must have four arms, the over/under reading must agree across eight test-time views, and two centre-line resolutions must give the same type. When a check fails, the system abstains and says why.

Our contributions are:

1. A structured formulation of knot recognition from images: dense crossing, over/under, mask and centre-line maps, read into a diagram whose topology is computed exactly (Sec. 3). It does not depend on the backbone: a 0.86M-parameter transformer and a pair of standard U-Nets both work, and the U-Nets are stronger (86% vs 69% answered, Tab. 1).
2. Consistency checks that convert errors into abstentions. Without test-time view agreement the model answers 12% of silhouettes, 6 of them wrongly; the Euler gate removes all 10 wrong answers of a skeleton parser. With all checks our system gives 0 wrong answers on 240 photo-like images and 60 silhouettes (Sec. 4 and Tab. 3).
3. KnotBench-Photo: renders with exact ground truth, cable- and session-disjoint splits, topology accuracy with abstention and cable-level bootstrap intervals, plus a protocol and annotation tool for real cable photographs (Sec. 5).
4. An analysis of the main failure mode, strands passing within a cable width: a full-resolution centre-line head raises coverage at 1.0–1.3 cable widths from 12% to 64%, and a first real-image test answers 12 of 166 pictures with no error (Sec. 6).
5. A blind evaluation protocol: a frozen model, held-out sets frozen by hash, and a ledger that refuses to score a second main model. On 308 blind drawings and renders the system answers 83 and errs on 5; we analyse these failures.

2. Related work

Knots in images and in data. Machine learning on knots has mostly used symbolic inputs: invariants and diagram codes [4, 5], with graph networks on diagrams [25], and grid diagrams have been classified with deep networks [13]. Closest to us, Dranowski *et al*. [7] predict crossing number from knot pictures with CNNs and transformers. They propose reconstructing PD codes and computing invariants as future work. We build that second stage: diagram reconstruction with over/under, exact identification and abstention, evaluated with cable-disjoint splits. Robotic rope manipulation reads rope state from images as keypoints or traced paths for grasping and untangling [10, 17, 24], not as a knot type. Exact identification uses SnapPy [3], KnotInfo [16] and knot Floer homology [20].

Curvilinear structure and line drawings. Vessel, road and neurite extraction recover thin structures and their connectivity [1, 21, 22]. Wireframe parsers predict junctions and line segments [12, 27], and technical-drawing vectorisation recovers strokes [8]. None of these recovers which strand lies on top at a crossing, which is what makes a diagram a knot.

Dense structured prediction. Our crossing head follows centre-heatmap detectors [18, 26] with a focal loss [15], and our backbone uses transformer blocks [6]. Foundation features [14, 19] are a natural alternative backbone, which we leave as a baseline to add.

Selective prediction. Abstaining classifiers trade coverage for risk [9], and calibration aligns confidence with accuracy [11]. Our abstentions come mostly from consistency checks grounded in planar topology, not from a learned confidence threshold.

3. From pixels to a knot diagram

Diagram model. A diagram is a set of closed strands (polylines in the image plane) and crossings. Each crossing has two passages: an over-strand and an under-strand. Its sign is the right-hand rule $\mathrm{sign}(\mathbf{t}_o \times \mathbf{t}_u)$ on the strand tangents, and the diagram's PD code follows by walking each strand. Identification is then exact: a hyperbolic exterior is matched by an isometry certificate, and knot Floer homology settles the remaining cases, including the unknot and chirality. An image fixes the diagram only if every crossing is visible and its over/under can be seen. A silhouette shows crossings but not over/under, so it fixes the projection but not the knot.

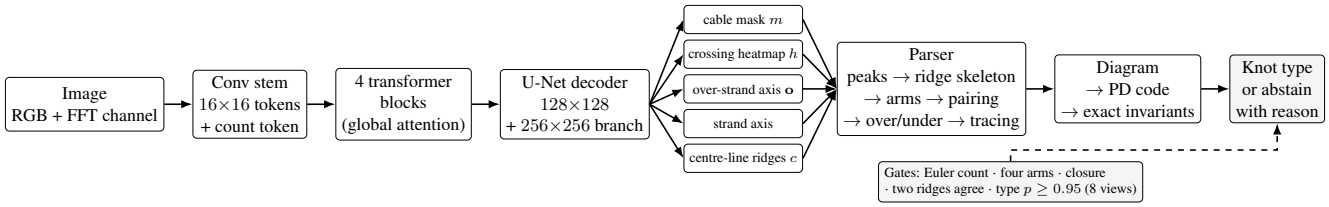


Figure 2. **Learn perception, compute topology.** The network predicts the parts of a diagram, not a label. A geometric parser assembles them, exact invariants identify the knot, and checks that do not trust the network decide whether to answer.

Input. The image is resized to 256×256 and given four channels: RGB and an FFT-cleaned grey channel (mirror-padded homomorphic filtering, which removes slow illumination, followed by Wiener shrinkage).

Networks. Any network that outputs the maps below can be used. We study two. (a) A NanoTransformer (0.86M parameters): a convolutional stem (stride 4) feeds a 16×16 token grid plus one count token into four transformer blocks (width 128, 4 heads). Global attention lets evidence from both sides of a crossing meet. A light U-Net decoder with stride-4 and stride-2 skips returns to a 128×128 grid, where five heads predict (Fig. 2):

- a soft cable mask m (fraction of each cell covered);
- a crossing heatmap h, trained with a centre-point focal loss;
- the over-strand axis $\mathbf{o} = (\cos 2\alpha, \sin 2\alpha)$ near each crossing;
- the strand axis elsewhere, used to pair arms;
- a centre-line ridge $c = \exp(-d^2/2\sigma^2)$, where d is the distance to the nearest strand centre-line.

A full-resolution branch (input-level convolutions fused with the decoder) predicts the same ridge on a 256×256 grid. On the coarse grid, two centre-lines one cable width apart are only 1.5–3 cells apart and merge. Axial (doubled-angle) targets make a strand and its reverse identical, which is all an image can show. Dihedral augmentations transform the axial targets consistently. (b) A pair of standard U-Nets (1.6M parameters each, 5 levels from 256^2 to 16^2): one predicts the mask and the centre-lines, the other the crossing heatmap with the over-strand axis as a per-keypoint attribute, as in CenterNet [26]. (c) A *UNetFormer* (2.9M parameters), a single network that predicts every map. It has a U-Net encoder and decoder with skip connections at all five resolutions, as in (b), and four transformer blocks (width 192, 6 heads) with a count token on the 16×16 grid at the bottleneck, as in TransUNet [2]. The transformer's output is added back to the bottleneck through a zero-initialised projection, so training starts from a plain U-Net.

Parser. (i) Crossings are heatmap peaks above 0.35, refined to sub-cell position. (ii) The skeleton is taken from the thresholded ridge. Specks below $(0.6w)^2$ pixels are removed, where w is the estimated cable width. Short gaps between facing ends are bridged, and the skeleton is cut on a disc around each crossing, leaving arcs whose ends are the crossing's arms. On cluttered backgrounds the networks draw short centre-line pieces on background texture. Of 134 free ends that made the NanoTransformer abstain on the benchmark's unseen backgrounds, 128 lay off the cable. A separate piece that carries no detected crossing and is open (its skeleton has an end) cannot belong to a closed diagram, so it is removed. Closed loops without crossings, such as an unknot or an unlinked component, are kept. (iii) Of the three ways to split four arms into two strands, we take the one whose strands run straightest. (iv) The strand whose axis agrees best with $\mathbf{o}$ is on top. Its probability is $p = \frac{1}{2} + \left(\sigma(\kappa\,\Delta) - \frac{1}{2}\right)\rho^{\gamma}$, where Δ is the margin between the two strands, κ is fitted on the training split, and ρ is the mean resultant length of $\mathbf{o}$ across eight test-time views ($\gamma = 2$, chosen on a development set). Where the views disagree, as on a silhouette, $\rho \to 0$ and $p \to \frac{1}{2}$. (v) Tracing arcs through crossings yields oriented strands and passages, i.e. a diagram.

Gap evidence (optional). View agreement cannot detect over/under evidence that every view misreads in the same way. The picture can be checked directly: the strand on top keeps its outline through the crossing, and the strand underneath loses it. For each strand through a crossing we sample, every half pixel of arc length t in the 512-pixel image, the contrast across both of its edges at $\pm w/2$ on the normal. The edge is *missing* where this contrast is below $\tau = 0.4$ times its median on the free part of the strand. The strand's interruption is a Lebesgue measure,

$$G = \tfrac{1}{w}\left|\{t \in [-T, T] : \text{edge missing at } t\}\right|, \qquad T = \tfrac{w}{2\sin\theta} + w,$$

where θ is the crossing angle. $D = G_{\text{under}} - G_{\text{over}}$ is the picture's evidence for the network's reading. It enters the log-odds, $\operatorname{logit} p' = \operatorname{logit} p + \lambda D$, and the reading flips if $p' < \frac{1}{2}$. $\lambda = 1.7$ maximises the likelihood of 856 matched crossings on development images; no test image is involved.

Minkowski attention and a Lebesgue loss. We test three changes to the NanoTransformer, each added to the previous one. (1) *Lebesgue (layer-cake) loss.* The parser thresholds the mask at levels $t \in [0.15, 0.85]$ and the ridges at 0.5. By the layer-cake formula, the mean over $t \sim U[a, b]$ of the area of $\{\hat{p} > t\} \triangle \{m > t\}$ equals $\mathbb{E}\,|\mathrm{clip}(\hat{p}, a, b) - \mathrm{clip}(m, a, b)|/(b - a)$. We add this term with weight 1 for the mask ($[0.15, 0.85]$) and both ridges ($[0.3, 0.7]$). It penalises exactly the pixels whose thresholded value depends on the level. (2) *Minkowski position bias.* Each head adds $-\alpha_h \|\mathbf{r}_i - \mathbf{r}_j\|_{p_h}$ to its logits, where $\mathbf{r}$ are token positions on the 16×16 grid. Both $p_h = 1 + \mathrm{softplus}(\rho_h)$ (initially 2) and α_h (initially 0.2 down to 0.01 across heads) are learned per block. The ℓ_p distance is invariant under the eight flips and quarter turns of the square, the group used for test-time views. (3) *Minkowski content attention.* The logits are $-t_h \|\mathbf{q}_i - \mathbf{k}_j\|_p^p / (p\sqrt{d})$, with $p = 1$ on two heads and $p = 2$ on the other two. For $p = 2$ this is dot-product attention less a key-norm term. A learned p would require a per-coordinate $|\mathbf{q} - \mathbf{k}|^p$ tensor, about 400 times slower on a CPU.

4. Consistency checks and abstention

The system reports a type only if all of the following hold; otherwise it abstains and names the failed check.

1. **Euler count.** The projection of k closed curves with n crossings is a 4-valent planar graph, so the ridge band has $n + \pi$ bounded faces, where π is the number of connected pieces. A missed crossing adds an unexplained face.
2. **Four arms.** Every detected crossing has exactly four arms. No skeleton branch point lies away from a detected crossing.
3. **Closure.** The traversal closes. Free ends require a recorded closure (a fixture, or ends joined), as for open cables.
4. **Two resolutions agree.** The diagram is parsed from both ridges. If both pass, they must give the same type.
5. **Type probability.** Uncertain crossings are enumerated over their two readings, weighted by p, and the most likely type must reach probability 0.95.

5. KnotBench-Photo

Synthetic split. Cables are random closed braids (2–4 strands, 2–9 generators) deformed in 3D and projected with a random rotation, so crossings, over/under and the type are exact. They are rendered as shaded tubes in five sessions: studio (S1), wood desk (S2), dim gradient (S3), cluttered texture (S4) and a silhouette diagnostic (S0) where over/under is invisible. The benchmark has 300 images of 60 cables. S3 and S4 backgrounds never appear in training.

Splits and metrics. Splits hold out whole cables and whole sessions; views of one cable never straddle splits. We report:

- crossing precision and recall (Hungarian matching within one cable width);
- over/under and pairing accuracy on matched crossings;
- answered fraction and number of wrong answers (topology accuracy counts abstentions as wrong);
- 95% intervals from a bootstrap over cables.

Close-strands stress set. The set has 100 renders binned by the smallest centre-line gap between strand pieces away from crossings, in cable widths (0.6–1.0, 1.0–1.3, 1.3–1.7, $\geq$1.7), 25 per bin. It is used for development only; its seeds are disjoint from training and the benchmark.

Real pictures. 166 public images of knots (109 drawings, 22 3D renders, 8 logos and 27 photographs of physical objects), labelled only by knot type. The measure is answered, correct (up to mirror image) or wrong.

Real cable photographs. We provide a capture protocol and a browser annotation tool. The protocol covers coax, rope and thread, four backgrounds, three lights and three views per session. The tool records crossings with over/under, visibility, free ends and the closure. A scoring script evaluates them with the same metrics after letterboxing, never stretching, the images. Report the real cable set here once collected.

6. Experiments

Training. The model (0.86M parameters) is first trained on 2,000 tube renders (plain and wood backgrounds only) for 14 epochs. It is then fine-tuned on 3,000 renders, 60% of them in broader styles: outlined and hollow ribbons, textbook diagrams with gaps, rope twist, shaded tubes, multicoloured links and widths of 5–32 px. Torus, Lissajous and random Fourier curves are added to braid closures. Finally the full-resolution ridge branch is trained for 4 epochs on the frozen trunk. The full recipe takes about 2.5 hours on two CPU cores; we report three seeds.

Main results. Table 1 compares label prediction, a classical parser and our formulation with three dense networks, all trained on the same renders. The 250-class classifier has median top-class probability 0.007; forced to answer it is right on 12% of images, and at $p \geq 0.95$ it answers none. The skeleton parser misses most crossings and reads over/under at chance; without the Euler gate it gives 10 wrong answers. A U-Net that only segments finds crossings as well as any model (precision 0.99, recall 0.997) yet names nothing: without over/under evidence no type

Method	Params	Cross. P	Cross. R	O/U	Answered	Unseen ans.	Wrong	Blind: ans. / wrong
Image-to-label classifier, forced	0.67M	–	–	–	100%	100%	210	308 / 306
Image-to-label classifier, $p \geq 0.95$	0.67M	–	–	–	0%	–	0	0 / 0
Skeleton traverser + gates	–	0.11	0.16	0.49	0%	–	0 (10 w/o gate)	–
Parser + U-Net segmentation only	1.6M	0.99	0.997	0.44	0%	0%	0	0 / 0
Parser + U-Net seg. + CenterNet keypoints	3.1M	1.00	0.98	1.00	86%	74%	0	138 / 9
Parser + NanoTransformer, mask skeleton (v1)	0.86M	0.93	0.90	1.00	52%	31%	0	–
Parser + NanoTransformer (v3, frozen)	0.86M	0.93	0.997	1.00	69%	40%	0	83 / 5
Parser + NanoTransformer, 3 seeds (mean±sd)	0.86M	–	0.997	–	68±3%	37±5%	0 total	84 / 5.3

Table 1. **Main results on KnotBench-Photo** (240 photo-like images of 60 cables; unseen = backgrounds S3/S4). Crossing precision (P) and recall (R), over/under (O/U) accuracy on matched crossings, fraction of images answered, and wrong answers. Our full model recovers 0.997 of crossings and answers 69% of images with 0 wrong answers.

reaches probability 0.95. Detecting crossings is not the hard part; reading over/under is. With over/under heads, both backbones read crossings and over/under almost perfectly and make no wrong call. The U-Net pair answers more (86% vs 69%), most clearly on unseen backgrounds (74% vs 40%). It has 3.6 times the parameters and was trained in one stage on the broader styles, while the NanoTransformer was first trained on plain tube renders; a size-matched comparison is left open. Per-session results are in the supplement.

Beating the U-Net pair. The U-Net pair's lead over the NanoTransformer does not come from crossings or over/under, which both read almost perfectly on the benchmark. It comes from the dense path. On cluttered backgrounds the NanoTransformer's centre-line and mask fire on background texture (Sec. 3). We therefore compare three models on equal terms (Tab. 2): the U-Net pair, the NanoTransformer widened to the same size and trained in one stage, and the UNetFormer. The widened NanoTransformer stays far behind (71.3 correct on blind v2), so size and training recipe do not explain the gap. The UNetFormer answers more blind-v2 images correctly than the U-Net pair at every seed (+12, +9, +7; 110.7 vs 101.3 on average). It also answers more real pictures (+4, +6, +8; 25.3 vs 19.3). Image by image, 43 blind-v2 readings are correct only for the UNetFormer and 15 only for the U-Net pair, summed over seeds (exact McNemar $p = 0.02, 0.06, 0.12$ per seed). On real pictures the split is 26 vs 9. Wrong answers barely change: 3 vs 2 on blind v2 over three seeds, all of them the same drawing of 9_25 read as 3_1 # 4_1 by both models. On the real pictures every error is on one picture, which the set holds twice: a single strand with three figure-eight lobes, read as 4_1 # 4_1 # 4_1 while its folder label is 4_1. On the benchmark the U-Net pair keeps a small edge (92.1% vs 90.4% answered, no wrong answer for either). On unseen backgrounds the per-seed difference is -10.8, +1.7, -0.8 points. The clDice topology loss [22] trades part of the UNetFormer's blind and real gain for benchmark coverage (91.5%; 104.3 correct on blind v2). The centre-line was not breaking, so a loss against breaks was not the missing piece.

Ablations. Table 3 isolates the parser's components.

- **Skeleton source.** Coverage comes from the skeleton source. Mask, then 128-grid ridge, then 256-grid ridge, then both ridges gives 52, 62, 68 and 69% answered, and 31, 28, 39 and 40% on unseen backgrounds.
- **Test-time views.** Without them, the model answers 12% of silhouettes, 6 of them wrongly: from one view it commits to an over/under reading it cannot see. Averaging the views without the consistency shrink ($\gamma = 0$) is worse: 62% of silhouettes answered, 25 wrongly. The shrink, not the averaging, is what makes the system abstain.
- **FFT input channel and seeds.** Three seeds of the full recipe give 68±3% answered (37±5% on unseen backgrounds) and 0 wrong answers in total. Retrained without the FFT channel (one seed) the model answers 63% (33% unseen): slightly lower, but within about two standard deviations of the seed spread, so we do not claim a benefit from the channel.
- **Clutter filter.** Removing open centre-line pieces that carry no crossing raises the frozen model's answered fraction from 69% to 82% (unseen backgrounds 40% to 68%) and the CPU-trained U-Net pair's from 86% to 92%, with no new wrong answers. The filter was designed after inspecting the benchmark's unseen-background failures and checked on 80 new renders in those styles, so its benchmark gain is not blind. On blind v2 it changes the frozen model by 0 images and the U-Net pair by 1.
- **Euler gate.** On renders the gate changes nothing for our model, since the other checks already catch its few misses. For the skeleton parser it removes all 10 wrong answers. We keep it as a network-independent safety net for real images, where detection is weaker.

Strands that touch. Abstentions on KnotBench-Photo come mostly from places where two strands pass within a cable width. Their masks merge, the mask skeleton grows

Model	Params	KnotBench-Photo ans.	KnotBench-Photo unseen	KnotBench-Photo wrong	Close ans. (wrong)	Real ans. (wrong)	Blind v2 corr. (wrong)
U-Net pair (seg. + keypoints)	3.1M	92.1±2.9%	85.8±5.8%	0	53.0±1.0% (6)	19.3±2.9 (2)	101.3±2.5 (2)
NanoTransformer, widened, one stage	2.9M	85.6±2.4%	72.8±5.4%	0	49.0±2.0% (5)	13.7±2.5 (1)	71.3±7.8 (4)
UNetFormer (ours)	2.9M	90.4±2.2%	82.5±4.4%	0	53.0±2.0% (5)	25.3±1.5 (3)	110.7±2.5 (3)
UNetFormer + clDice	2.9M	91.5±3.2%	84.7±6.3%	0	55.3±1.5% (6)	22.7±2.1 (2)	104.3±4.5 (3)

Table 2. **Matched comparison with the U-Net pair.** Same data (3,000 renders), same one-stage recipe (12 epochs), same device (Apple GPU), and the same parser (with the clutter filter). Mean±sd over three seeds; wrong answers are summed over seeds. Blind v2 has 312 images, real 166.

Variant	Cross. R	Answered	Unseen	Wrong	Silh. ans.
Full (two ridges, Euler gate, 8-view TTA)	0.997	69%	40%	0	0%
256-grid ridge only	0.997	68%	39%	0	0%
128-grid ridge only	0.997	62%	28%	0	0%
without Euler gate	0.997	70%	42%	0	0%
without test-time views	0.998	57%	22%	0	12% (6 wrong)
views averaged, no consistency shrink ($\gamma = 0$)	0.997	69%	40%	0	62% (25 wrong)
Mask skeleton, no ridge (v1 model)	0.90	52%	31%	0	0%
without FFT input channel (retrained, full recipe)	0.996	63%	33%	0	0%

Table 3. **Ablations** on KnotBench-Photo (v3 trunk unless noted). Silh. ans. = fraction of silhouettes answered, which must be 0 because over/under is not visible.

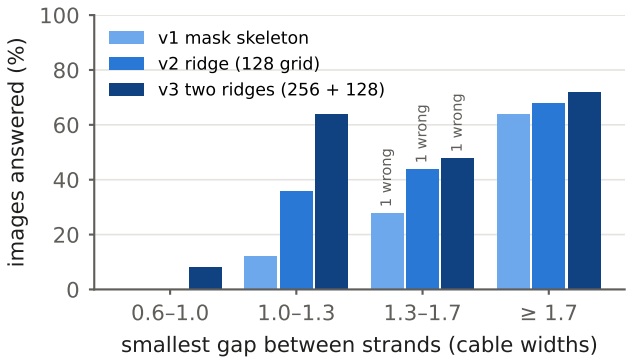


Figure 3. **Coverage as strands come closer** (100 images, 25 per bin). The full-resolution centre-line head keeps strands apart down to about one cable width. Below that, the strands touch and the system abstains. The single wrong answer (same image in every version) is a 2.7 px double crossing that no single view resolves.

a false branch, and the face between them disappears from the Euler count. Detection and over/under are not the problem: crossing recall stays 0.93–0.97 and over/under accuracy 0.98–1.00 in every bin. Figure 3 shows that the ridge skeleton raises overall coverage on the stress set from 26% to 48%. At 1.0–1.3 cable widths it rises from 12% to 64%. Below one cable width it reaches only 8%: there the strands touch, and abstaining (and asking for another view) is the intended behaviour.

Real pictures. On 166 real knot pictures (Tab. 4), the skeleton parser answers none. Our model answers 5 before and 12 after the broader styles, all correct. Answers come

Category	n	Traverser	v1	v2	v3
Drawing	109	0	3	4	4
3D render	22	0	2	7	8
Logo	8	0	0	0	0
Photo of an object	27	0	0	0	0
All (wrong)	166	0	5 (0)	11 (0)	12 (0)

Table 4. **Real knot pictures answered** (166 public images, labelled by type only); wrong answers in brackets. Traverser = skeleton parser.

Method	Drawings (knotR) ans.	Drawings (knotR) correct	Drawings (knotR) wrong	Renders (POV-Ray) ans.	Renders (POV-Ray) correct	Renders (POV-Ray) wrong
Classifier, forced	98	0	98	210	2	208
Classifier, $p \geq 0.95$	0	0	0	0	0	0
U-Net segmentation + parser	0	0	0	0	0	0
U-Net seg. + keypoints + parser	94	86	8	44	43	1
Ours (frozen model)	42	37	5	41	41	0

Table 5. **Blind set**, scored once with the frozen model (308 images from sources never used or viewed during development: knotR drawings and POV-Ray renders of KnotPlot centrelines). Correct = same type up to mirror image.

from drawings and renders. None of the 27 photographs of physical objects (rope, cord, sculpture) is answered: their textures still break the skeleton. The training styles were chosen after inspecting four failures from this set, so these numbers are not a blind test. Add a blind real set and the annotated real cable photographs.

Blind evaluation. Before scoring, the NanoTransformer and its parser settings were frozen by hash, and the set was built without viewing any image (Tab. 5). Renders behave like the benchmark: 41 of 210 answered, 0 wrong. Drawings expose a failure mode the benchmark never showed: 42 of 98 answered and 5 wrong (Fig. 4). In each of them the under-strand gap is one or two pixels wide, every crossing is read with probability 0.96–0.99, and all eight test-time views agree on the reading. View agreement detects evidence that depends on the view (a silhouette), not evidence that is consistently misread. A network-independent check of the intensity break at each crossing is the obvious next

gate; it must be evaluated on a new blind set.

Baselines on the blind set. The U-Net pair, scored once as a baseline, answers 138 images, 129 correctly; 9 of its answers are wrong, 8 of them on drawings with nearly invisible gaps, the same failure as the frozen model. The classifier is right on 0.6% of the blind set (and on 18% of the real pictures it was trained on).

Minkowski attention and the Lebesgue loss. Each variant was trained with the full recipe (Tab. 6). The content-attention variant has three CPU seeds: 0 and 1 on the machine that trained the plain model, and 2 on a laptop CPU. All three variants also have three seeds trained on an Apple GPU. A seed fixes the data order and augmentation, so we compare each variant with the plain model of the same seed. Against it, Minkowski content attention changes the answered fraction on unseen backgrounds by +0, +15, +15 points on CPU, and the number of correct blind-v2 images by +5, +1, +5. It makes one more wrong blind-v2 answer over the three seeds (3 vs 2). No paired comparison goes the other way. On the GPU the differences are larger (+4, +8, +22 points; +8, +6, +10 images). However, the plain model was never trained on the GPU, and the Lebesgue loss alone, which has no Minkowski component, gains about as much there (-1, +12, +15 points). The GPU rows therefore mix the device with the model. On CPU at seed 0, the Lebesgue loss alone loses 5 points on unseen backgrounds and the position bias gains 3. On the 166 real pictures there is no consistent change (+5, -1, -1). No model makes a wrong benchmark call. On the close-strands set the variant makes 4 wrong answers over three seeds, against 3 for the plain model. The learned exponents stay Euclidean: $p \in [1.89, 2.10]$ in every head of all 10 trained models, and the slopes stay near their initial values (0.009–0.23). Whatever helps is a multi-scale locality prior on the token grid, not a non-Euclidean neighbourhood. Seeds alone move blind-v2 by up to three images (51–54 correct for the plain model). Three paired seeds give a consistent direction (benchmark coverage and blind v2 up, wrong answers unchanged), not a precise effect size. The variants stay well behind the U-Net pair.

Gap evidence and a second blind set. Blind v1 can no longer test a remedy for its own failures, because they were inspected. We therefore froze a second blind set (Tab. 7) before scoring the gap evidence or any new model on it. Its drawings come from a renderer used nowhere in training, with gaps from clear to nearly invisible. Its renders show the v1 curves from new directions, which gives new diagrams. The three plain seeds answer 51–54 images correctly (mean 52.3). With the gap evidence, the frozen network answers 57 correctly (54 without it) with 1 wrong answer, as before. The gain comes from drawings (26 → 30 correct). On the benchmark, the close-strands set and the real pictures, nothing changes (Tab. 6). On development images the evidence improves the log-likelihood of the network's over/under readings from -155 to -151 and adds one answer (135 → 136, no wrong). A veto rule ($p \to \frac{1}{2}$ when D is small) instead removes correct answers (131 answered) and was not adopted.

Two caveats. First, a cross-section measure (correlation of the strand's profile with its free profile) fits the development crossings better (log-likelihood -142). But on the shaded 3D renders of the real-picture set it has the wrong sign at most crossings the network reads correctly (all seven on one 7_1 render), and it cost six correct answers there. We chose the edge measure after seeing this, so the real-picture numbers for the gap evidence are not blind. Blind v2 was frozen before that choice was scored. Second, the effect is small: three answers out of 312, just above the seed-to-seed range. On blind v1 (post hoc) the evidence adds six correct answers and one wrong, and the original five errors remain: they are not over/under errors alone, as 7_1 read as 3_1 shows. The U-Net pair is far stronger on v2 (95 correct of 96 answered, 1 wrong), as on every other set.

Runtime. On four CPU threads of a laptop (Apple silicon, PyTorch on CPU), the median image takes 0.54 s in the network with eight test-time views (0.08 s with one) and 0.35 s in the parser, including both skeleton sources and the gap measure: 0.9 s in total (20 benchmark images).

7. Limitations

Both backbones make confident errors on the blind set (5 and 9), almost all on drawings with nearly invisible gaps: confidence is calibrated against view-dependent ambiguity, not against consistent misreading. The picture's own gap evidence adds answers on a second blind set without adding errors, but it does not remove those errors. The Minkowski variants rest on three paired seeds per device, and the plain model was not trained on the GPU, so their effect size is uncertain. All quantitative cable results are on renders. No annotated photographs of real cables exist yet. On public pictures the system answers drawings and renders but no photograph of a physical object, and those pictures informed our choice of training styles. Strands that touch (gap below one cable width) and double crossings smaller than a few pixels cannot be resolved from one view; multi-view fusion is implemented but untested on real data. The output is a knot type, not a physical untangling plan: friction, stiffness and tension decide how hard a cable is to undo.

8. Conclusion

Reading the diagram and computing the topology, instead of classifying the image, gives a knot recogniser that gener-

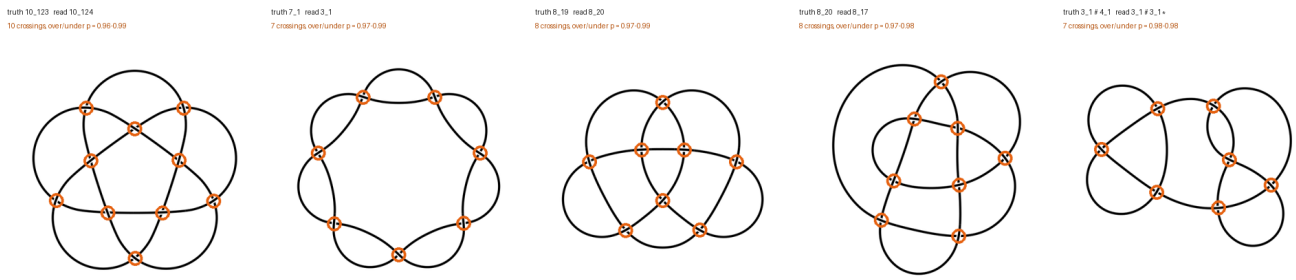


Figure 4. **All wrong answers on the blind set** (post-scoring analysis). Orange circles: detected crossings. Gaps at the under-strands are one or two pixels wide; the model reads over/under with probability 0.96–0.99 at every crossing and all test-time views agree.

Model	KnotBench-Photo ans.	unseen	wrong	Close ans. (wrong)	Real ans. (wrong)	Blind v1† corr. / wrong	Blind v2 corr. / wrong
Plain model, 3 seeds	68±3%	37±5%	0	49±2% (3)	12±1 (2)	78±12 / 16	52±2 / 2
+ Lebesgue (layer-cake) loss, seed 0	66%	35%	0	48% (1)	8 (0)	96 / 7	56 / 0
+ Minkowski position bias, seed 0	70%	43%	0	48% (2)	14 (0)	82 / 6	60 / 1
+ Minkowski content attention, 3 seeds	72±4%	47±7%	0	49±2% (4)	13±4 (1)	80±2 / 17	56±3 / 3
trained on an Apple GPU (MPS); the plain model above was trained on CPU							
+ Lebesgue (layer-cake) loss, 3 seeds	71±4%	46±8%	0	–	13±5 (1)	–	55±3 / 2
+ Minkowski position bias, 3 seeds	73±3%	49±7%	0	–	12±5 (0)	–	55±6 / 3
+ Minkowski content attention, 3 seeds	73±6%	48±12%	0	–	12±3 (0)	–	60±2 / 3
Frozen model + gap evidence (inference)	69%	40%	0	48% (1)	12 (0)	84 / 6	57 / 1
U-Net pair (baseline)	86%	74%	0	50% (1)	15 (0)	129 / 9	95 / 1

Table 6. **Minkowski attention, the Lebesgue loss and gap evidence.** Mean±sd over seeds; wrong answers are summed over seeds. Variants are added cumulatively (Sec. 3). A seed fixes the data order and augmentation, so each variant has a plain counterpart with the same seed (see text). †Blind v1 is no longer blind for the gap evidence, whose design followed its errors. Blind v2 is (Tab. 7).

Model	Drawings (halo gaps) ans.	correct	wrong	Renders (new views) ans.	correct	wrong
Plain, seed 0 (frozen model)	26	26	0	29	28	1
Plain, seed 1	21	21	0	31	31	0
Plain, seed 2	27	26	1	25	25	0
Plain without FFT channel	18	17	1	32	32	0
+ Lebesgue loss (seed 0)	26	26	0	30	30	0
+ Minkowski position bias (seed 0)	25	24	1	36	36	0
+ Minkowski content attention (seed 0)	25	24	1	35	35	0
Frozen model + gap evidence	30	30	0	28	27	1
U-Net pair (baseline)	47	46	1	49	49	0

Table 7. **Blind set v2** (312 images, frozen before any model or setting in this table was scored on it, never viewed: 105 line drawings with halo gaps of 0.1–0.8 line widths, 207 POV-Ray renders from new viewpoints). Correct = same type up to mirror image.

alises past a census, explains its answers and abstains when the image is not enough. The gain comes from the formulation, not from a particular backbone: standard U-Nets do better than our first transformer, and a U-Net with a transformer bottleneck does better than both on blind and real pictures. On renders it reads crossings and over/under almost perfectly and makes no wrong call. The open problem is perception on real photographs, for which we release the benchmark protocol, annotation tool and scoring code.

References

[1] Favyen Bastani, Songtao He, Sofiane Abbar, Mohammad Alizadeh, Hari Balakrishnan, Sanjay Chawla, Sam Madden, and David DeWitt. RoadTracer: Automatic extraction of road networks from aerial images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4720–4728, 2018. 2

[2] Jieneng Chen, Yongyi Lu, Qihang Yu, Xiangde Luo, Ehsan Adeli, Yan Wang, Le Lu, Alan L. Yuille, and Yuyin Zhou. Transunet: Transformers make strong encoders for medical image segmentation. *arXiv preprint arXiv:2102.04306*, 2021. 3

[3] Marc Culler, Nathan M. Dunfield, Matthias Goerner, and Jeffrey R. Weeks. SnapPy, a computer program for studying the geometry and topology of 3-manifolds. `https://snappy.computop.org`. 2

[4] Alex Davies, András Juhász, Marc Lackenby, and Nenad Tomasev. The signature and cusp geometry of hyperbolic knots. *arXiv preprint arXiv:2111.15323*, 2021. 2

[5] Alex Davies, Petar Veličković, Lars Buesing, et al. Advancing mathematics by guiding human intuition with AI. *Nature*, 600:70–74, 2021. 2

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Syl-

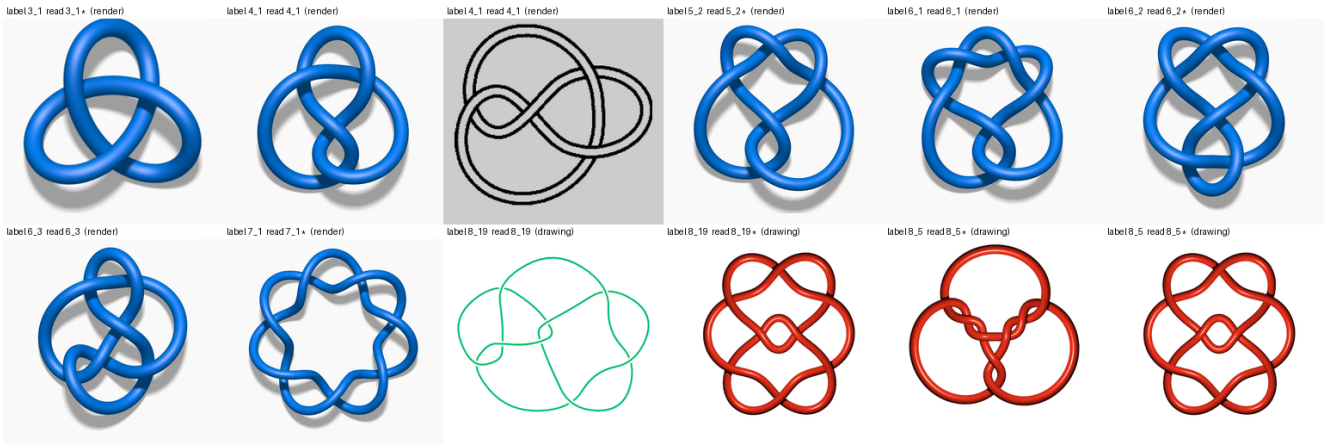


Figure 5. **All 12 real pictures our system answers**, each correct (up to mirror image, marked $*$). Every answer comes from a drawing or a render; none of the photographs of physical objects is answered.

vain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. 2

[7] Anne Dranowski, Yura Kabkov, and Daniel Tubbenhauer. On knot detection via picture recognition. arXiv preprint arXiv:2510.06284, 2025. 2

[8] Vage Egiazarian, Oleg Voynov, Alexey Artemov, Denis Volkhonskiy, Aleksandr Safin, Maria Taktasheva, Denis Zorin, and Evgeny Burnaev. Deep vectorization of technical drawings. In *Computer Vision – ECCV 2020*, pages 582–598. Springer, 2020. 2

[9] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, pages 4878–4887, 2017. 2

[10] Jennifer Grannen, Priya Sundaresan, Brijen Thananjeyan, Jeffrey Ichnowski, Ashwin Balakrishna, Vainavi Viswanath, et al. Untangling dense knots by learning task-relevant keypoints. In *Proceedings of the 2020 Conference on Robot Learning*, pages 782–800, 2021. 2

[11] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1321–1330, 2017. 2

[12] Kun Huang, Yifan Wang, Zihan Zhou, Tianjiao Ding, Shenghua Gao, and Yi Ma. Learning to parse wireframes in images of man-made environments. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 626–635, 2018. 2

[13] Louis H. Kauffman, Nikolay E. Russkikh, and Iskander A. Taimanov. Rectangular knot diagrams classification with deep learning. *Journal of Knot Theory and Its Ramifications*, 31(11):2250067, 2022. 2

[14] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4015–4026, 2023. 2

[15] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017. 2

[16] Charles Livingston and Allison H. Moore. Knotinfo: Table of knot invariants. https://knotinfo.math.indiana.edu. Accessed 2026-10-13. 2

[17] Ashvin Nair, Dian Chen, Pulkit Agrawal, Phillip Isola, Pieter Abbeel, Jitendra Malik, and Sergey Levine. Combining self-supervised learning and imitation for vision-based rope manipulation. In *IEEE International Conference on Robotics and Automation*, 2017. 2

[18] Alejandro Newell, Kaiyu Yang, and Jia Deng. Stacked hourglass networks for human pose estimation. In *Computer Vision – ECCV 2016*, pages 483–499. Springer, 2016. 2

[19] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. 2

[20] Peter Ozsváth and Zoltán Szabó. Holomorphic disks and knot invariants. *Advances in Mathematics*, 186(1):58–116, 2004. 2

[21] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pages 234–241. Springer, 2015. 2

[22] Suprosanna Shit, Johannes C. Paetzold, Anjany Sekuboyina, Ivan Ezhov, Alexander Unger, Andrey Zhylka, Josien P. W. Pluim, Ulrich Bauer, and Bjoern H. Menze. clDice – a novel topology-preserving loss function for tubular structure segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16560–16569, 2021. 2, 5

[23] William R. Taylor. A deeply knotted protein structure and how it might fold. *Nature*, 406(6798):916–919, 2000. 1

[24] Vainavi Viswanath, Kaushik Shivakumar, Mallika Parulekar, Jainil Ajmera, Justin Kerr, Jeffrey Ichnowski, et al. HANDLOOM: Learned tracing of one-dimensional objects for inspection and manipulation. In *Proceedings of the 7th Conference on Robot Learning*, pages 341–357, 2023. 2

[25] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019. 2

[26] Xingyi Zhou, Dequan Wang, and Philipp Krähenbühl. Objects as points. arXiv preprint arXiv:1904.07850, 2019. 2, 3

[27] Yichao Zhou, Haozhi Qi, and Yi Ma. End-to-end wireframe parsing. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 962–971, 2019. 2